

TP - Comparaison d'algorithmes de tri

Soit une liste L de taille N où chaque cellule $L[i]$ avec $0 \leq i \leq N-1$ contient un nombre entier. Trier L par ordre croissant consiste à réordonner les valeurs des cellules de manière à ce que, quels que soient i et j , on a : $0 \leq i < j \leq N-1$ et $L[i] \leq L[j]$.

Pour passer d'une liste non triée à une liste triée par ordre croissant, il existe plusieurs méthodes de tri. L'objectif de ce TP est de programmer quatre de ces méthodes pour pouvoir ensuite comparer leur efficacité.

Environnement de programmation

Pour ce TP, vous utiliserez l'environnement Colab de Google :

<https://colab.research.google.com/>

L'ensemble du TP se fera dans un même notebook que vous appellerez **analyse_tris**

Exercice 1 - Création d'une liste

Créer une fonction **generer_liste** qui prend en paramètre un entier **nb_valeurs** indiquant le nombre de valeurs à mettre dans la liste, et qui retourne la liste créée. Les valeurs insérées dans la liste seront des valeurs entières générées aléatoirement sur l'intervalle $[1 ; \text{nb_valeurs} \times 10]$.

Exercice 2 - Tri itératif par sélection

Le tri par sélection consiste à chercher le plus petit élément de la liste et à le placer en première position. Une fois le plus petit élément positionné en première position, on recommence la même opération en commençant à la deuxième position de la liste : on cherche le plus petit élément compris dans la liste entre la 2e position et la fin de la liste, puis on place cet élément à la deuxième position. On recommence ainsi en partant de la 3e position, puis de la 4e et ainsi de suite jusqu'au bout de la liste.

Ecrire la fonction **trier_par_selection** qui prend en paramètre une liste et retourne cette même liste triée par ordre croissant.

Exercice 3 - Tri itératif par insertion

Le tri par insertion consiste à classer les deux premiers éléments de la liste. Une fois que les deux premiers sont ordonnés, on prend l'élément qui suit et on le classe à son tour dans ce qui a déjà été classé. Pour chaque élément i de la liste, on sait que les éléments positionnés aux indices de 0 à $i-1$ sont déjà classés. On va chercher la position j parmi les $i-1$ premiers éléments de manière à ce que $L[i] < L[j]$. On insère alors $T[i]$ à la position j .

Ecrire la fonction **trier_par_insertion** qui prend en paramètre une liste et retourne cette même liste triée par ordre croissant.

Exercice 4 - Tri récursif par fusion

Le tri par fusion est un algorithme qui propose de :

- Découper la liste en 2 parties de longueur égale (à un élément près) ;
- Trier les deux sous listes ;
- Fusionner ces deux sous listes triées pour n'en former qu'une seule qui soit elle-même triée.

Avant de réaliser la fonction de tri en elle-même, écrire une fonction **fusionner_listes** qui prend en paramètres 2 listes triées et retourne une liste triée issue de la fusion des deux listes passées en paramètres.

Ecrire la fonction **trier_par_fusion** qui prend en paramètre une liste et retourne cette même liste triée par ordre croissant.

Exercice 5 - Tri récursif rapide

Le tri rapide est un algorithme de tri très utilisé pour sa relative simplicité et sa rapidité.

Il se réalise en 4 étapes :

- Choisir un élément de la liste qu'on appelle pivot ; Cet élément peut être le premier élément de la liste, l'élément avec la valeur médiane, etc...
- Partitionner la liste en deux sous-listes : une sous-liste contenant l'ensemble des éléments qui sont inférieurs au pivot et une seconde sous-liste avec l'ensemble des éléments qui sont supérieurs au pivot ;
- Trier ces 2 sous-listes suivant la même méthode ;
- Concaténer ces 2 sous-listes triées sans oublier d'ajouter le pivot entre les 2.

Ecrire la fonction **partitionner_liste** qui prend en paramètres la valeur du pivot ainsi que la liste à partitionner, et qui retourne les deux sous-listes.

Ecrire la fonction **trier_rapide** qui prend en paramètre une liste et retourne cette même liste triée par ordre croissant.

Exercice 6 - Comparaison des différents tris

La comparaison des tris consistera à trier une même liste avec chacune des quatre méthodes de tris, et à calculer le temps nécessaire pour réaliser chaque tri. Pour que la comparaison entre ces différentes méthodes soit pertinente, vous devrez comparer 10 fois chaque méthode de tri avec une liste de taille N.

Pour analyser l'évolution des algorithmes de tri en fonction de la taille de la liste, vous commencerez par comparer les méthodes de tri sur des listes de 500 éléments, puis sur des listes de 1000 éléments et ainsi de suite jusqu'à trier des listes de 10 000 éléments en augmentant à chaque fois de 500 éléments.